

Object Oriented Programming

What are object oriented concepts? What is difference between object-based, object-oriented and fully object-oriented language?

- Pillars of Object Oriented Programming ... Grady Booch (Object Oriented Analysis Design)
 - Major Pillars ... Must be supported by any Object Oriented language
 - Abstraction
 - Encapsulation
 - Hierarchy ... Reusability
 - is-a (inheritance)
 - Person <|--- Employee <|--- Manager
 - has-a (association)
 - Computer <>--- Motherboard <>--- Processor
 - use-a (dependency)
 - MyLinkedList "use a" Stack class for reversing list.
 - creates-a (instantiation)
 - SessionFactory creates Session.
 - Modularity
 - Divide source code into multiple logical units ... better maintainability
 - Division
 - Source code files
 - Classes
 - Packages, Namespaces, Modules
 - Minor Pillars ... May be supported by any Object Oriented language
 - Typing
 - Strong vs Weak -- Data types
 - Java: Strong typing
 - JS: Weak typing
 - Static vs Dynamic -- Polymorphism
 - Compile time

- Runtime
 - Concurrency
 - Object behaviour in case of multi threading (how it work with race condition?)
 - Persistence
 - Maintaining state of object outside its scope (e.g. database, file - serialization, network)
- Object Based
 - Supports only abstraction & encapsulation (i.e. classes/objects)
 - e.g. Visual Basic, Old Java script, ...
- Object Oriented
 - Must support all major Pillars
 - e.g. C++, Java
- Fully Object Oriented
 - Must support all major and minor Pillars
 - Each data must be object (even primitive types must be objects)
 - No global variables/functions
 - e.g. C#

What is abstraction and encapsulation. Give real-life example.

- Abstraction
 - Getting essential details of the system.
 - Hides unnecessary (from user perspective) details.
 - Depends on perspective of user.
 - Outer view of the system.
 - Implemented with abstract classes and interfaces.
- Encapsulation
 - Binding code and data together in a class.
 - Inner view of the system.
 - Hiding data to achieve security.
 - Abstraction & Encapsulation are complementary to each other.
 - Implemented with private, protected members.
- Implementation

```
interface Account {
    public void deposit(double amount);
}
class BankAccount implements Account {
    private double balance;
    public double getBalance() {
        return this.balance;
    }
    public void deposit(double amount) {
        this.amount += balance;
    }
}
```

Feature	Abstraction	Encapsulation
Definition	Hiding unnecessary details and exposing only relevant features.	Hiding data and restricting direct access by binding it with methods.
Focus	Focuses on design – what should be exposed.	Focuses on implementation – how data should be protected.
Purpose	Simplifies complexity by providing a clear interface.	Protects data by preventing unintended modifications.
How is it achieved?	Using abstract classes, interfaces (OOP) or modular design (functional programming).	Using private/protected members and public methods (getters/setters) .
Perspective	Outer view (What the user sees).	Inner view (How data is stored and manipulated).
Example	A car driving system hides complex mechanics but provides a simple interface (steering, brake, accelerator).	The car engine encapsulates internal components to ensure safe operation.

What are advantages of Object Oriented Programming?

- Data security

```
struct Time {  
    int hour, minute, seconds;  
};  
// ....  
struct Time t1;  
t1.seconds = 100; // no compiler error -- may cause business logic to fail  
// ...  
incTime(&t1);
```

```
class Time {  
    private int hour, minute, seconds;  
    // methods ... setters/getters  
    public void setSeconds(int sec) {  
        if(sec < 0 || sec >= 60)  
            throw exception;  
        seconds = sec;  
    }  
};  
// ....  
struct Time t1;  
t1.setSeconds(100); // throw exception  
// ...  
t1.incTime();
```

- Modularity
 - Namespaces/Packages/Modules
 - Classes
- Reusability (Association & Inheritance)
 - Classes are reused

What is class and object? Give real-life example.

- Class is blue-print of the object. Class has fields (data) and methods (operations).
- Object is an instance of the class. One class can have multiple objects.
- Class is a logical entity.
- Object is a "physical" entity. Allocates memory.

What are characteristics of object? Explain them.

- State -- Values of data members
- Behaviour -- Operations performed/Methods called on state
- Identity -- Uniqueness of the object (e.g. address)

What is the need of getter and setter functions in class?

- Controlled access to the private fields
 - Programmer can choose which fields need to provide getter, setter or both.
 - Programmer can add validation logic into setter methods.

What is polymorphism? What are its types? Explain them with examples. What is the difference between function overloading and function overriding?

- Polymorphism -- Taking many forms.
 - Same name -- Different implementations
- Compile time/Static Polymorphism
 - Function with same name but different arguments in same scope -- Function overloading.
 - Internally compiler assigns different name to each method (Name mangling).
 - Compiler decides which function to be called (Early binding), depending on number/type of arguments.

```
int add(int a, int b) { // addii
    return a + b;
}
int add(int a, int b, int c) { // addiii
```

```
    return a + b + c;  
}
```

```
res = add(11, 22);  
res = add(11, 22, 33);
```

- False polymorphism
- Run time/Dynamic Polymorphism
 - Function with same name and same arguments redefined in derived class -- Function overriding.
 - Internally uses method table/virtual function table -- dynamic method dispatch.
 - Function to be called is decided at runtime (Late binding), depending on type of object (not the reference).

```
class Base {  
    public void print() {  
        System.out.println("Base.print() called");  
    }  
}  
  
class Derived extends Base {  
    public void print() {  
        //super.print(); // Base::print();  
        System.out.println("Derived.print() called");  
    }  
}
```

```
Base obj1 = new Base();  
obj1.print(); // call Base.print()  
Base obj2 = new Derived();  
obj2.print(); // call Derived.print()
```

- True polymorphism

What is function overloading? Which are the rules of function overloading? Why return type is not considered in function overloading?

- Function with same name but "different" arguments in same scope -- Function overloading.
- Arguments should differ in count, data type, and/or order.
- Return type is not considered, because collecting return value is not compulsory. Compiler will not be able to resolve the function to be called.

```
int divide(int a, int b) {  
    return a / b;  
}  
  
double divide(int a, int b) {  
    return (double)a / b;  
}
```

```
int r1 = divide(15, 5);  
double r2 = divide(15, 5);
```

```
divide(15, 5);
```

What are different types of hierarchy? When to use which one?

- is-a hierarchy -- Inheritance (kind of)
 - Inheritance (Generalization -> Specialization)
 - Student is a person.
 - ~~Student has a person.~~
- has-a hierarchy -- Association (part of)
 - Represents a whole-part relationship where one object contains another.

- Achieved via association, which can be:
 - Composition (Strong ownership, lifecycle dependency)
 - Aggregation (Weaker relationship, independent lifecycle)
- Employee has a Joining date.
- ~~Employee is a Joining date.~~
- use-a hierarchy -- Dependency
 - Represents a temporary relationship where an object depends on another to perform an action.
 - Achieved via method parameters, function calls, or local objects.
 - When an external service or utility class is used.
 - Document uses Printer (to print).
- creates-a hierarchy -- Instantiation/Factory
 - Represents an object-creation relationship, where one class instantiates another.
 - ProductFactory creates Product.

Why constructor is considered special member function of the class?

- Constructor has same name as of class name.
- Constructor doesn't have any explicit return type.
- Constructor is automatically called when object is created.

What is object slicing? Explain object slicing in context of upcasting?

- Assigning derived class object address to base class reference is called as "Up-casting".
- When derived class object is assigned to base class reference, through that base class reference one can access only the base class members. This is called "Object slicing".

```
Person p = new Employee();  
System.out.println(p.getName());  
System.out.println(p.getSalary()); // not allowed -- compiler error
```

What is down-casting and when it is required?

- Assigning base class reference to derived class reference is called as "Down-casting".

```
void displayStudent(Person p) {  
    System.out.println(p.getName());  
    Student s = (Student)p; // Down-casting  
    System.out.println(s.getMarks()); // allowed  
}
```

```
displayStudent(new Student(...));
```

```
void displayEmployee(Person p) {  
    System.out.println(p.getName());  
    Employee e = (Employee)p; // Down-casting  
    System.out.println(e.getSalary()); // allowed  
}
```

```
displayEmployee(new Employee(...));
```

```
void display(Person p) {  
    System.out.println(p.getName());  
    if(p instanceof Employee) {  
        Employee e = (Employee)p; // Down-casting  
        System.out.println(e.getSalary()); // allowed  
    }  
    if(p instanceof Student) {  
        Student s = (Student)p; // Down-casting  
    }  
}
```

```
        System.out.println(s.getMarks()); // allowed
    }
}
```

```
display(new Person(...));
display(new Student(...));
display(new Employee(...));
```

What do you know about association, composition and aggregation. Explain with the help of example.

- has-a relationship -- Association
- Composition (Tight coupling) -- part of

```
class engine {
    // ...
};
class car {
    engine e; // tight coupling
    // ...
};
```

- Aggregation (Loose coupling)

```
class driver {
    // ...
};
class car {
    driver *d; // loose coupling
```

```
// ...  
};
```

What are different types of inheritance? Explain with the help of example. What are problems with multiple inheritance?

- Inheritance represents: is-a relation.
- Generalization to Specialization.
- Person --> Employee --> Manager --> HRManager
- Person --> Student
- Types
 - Single inheritance
 - Multiple inheritance
 - Multi-level inheritance
 - Hierarchical inheritance
 - Hybrid inheritance
- Problems in multiple inheritance: Ambiguity

```
class bird {  
public:  
    void fly() {  
        // ...  
    }  
};  
class aeroplane {  
public:  
    void fly() {  
        // ...  
    }  
};  
class birdplane: public bird, public aeroplane {  
};
```

```
birdplane b;  
b.fly(); // Ambiguity: bird::fly() or aeroplane::fly() --> Compiler Error  
// b.bird::fly(); --> bird::fly()  
// b.aeroplane::fly(); --> aeroplane::fly()
```

Which are the different types of design pattern? Explain singleton design pattern.

- Design pattern is standard solutions to well-known problems in software development/coding.
- Design patterns are categorized into **three main types**, as defined in the **Gang of Four (GoF)** book "*Design Patterns: Elements of Reusable Object-Oriented Software*".
 1. **Creational Patterns** → Object creation mechanisms.
 2. **Structural Patterns** → Composition of classes and objects.
 3. **Behavioral Patterns** → Communication between objects.

1. Creational Design Patterns (Managing Object Creation)

These patterns deal with **object instantiation** to provide flexibility and reusability.

Pattern	Purpose
Singleton	Ensures only one instance of a class exists.
Factory Method	Creates objects without specifying exact class.
Abstract Factory	Provides an interface for creating related objects.
Builder	Constructs complex objects step by step.
Prototype	Creates new objects by copying an existing object.

Example: Singleton Pattern (Ensures a single instance of a class)

```
class Singleton {
    private static Singleton instance; // Static instance
    private Singleton() {} // Private constructor

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

2. Structural Design Patterns (Organizing Object Relationships)

These patterns **deal with class and object composition** to ensure flexible structures.

Pattern	Purpose
Adapter	Converts interface of a class into another expected interface.
Bridge	Separates abstraction from implementation.
Composite	Composes objects into tree structures.
Decorator	Adds responsibilities dynamically to objects.
Facade	Provides a simplified interface to a complex system.
Flyweight	Shares object instances to save memory.
Proxy	Acts as a placeholder for another object.

Example: Adapter Pattern (Converting one interface to another)

```
// Existing interface
interface OldCharger {
    void chargeWithMicroUSB();
}

// New interface
interface NewCharger {
    void chargeWithUSB_C();
}

// Adapter class to make old charger compatible with new interface
class ChargerAdapter implements NewCharger {
    OldCharger oldCharger;

    public ChargerAdapter(OldCharger oldCharger) {
        this.oldCharger = oldCharger;
    }

    @Override
    public void chargeWithUSB_C() {
        oldCharger.chargeWithMicroUSB(); // Adapting old method
    }
}
```

3. Behavioral Design Patterns (Managing Object Interactions)

These patterns **focus on communication and behavior** among objects.

Pattern	Purpose
Observer	Notifies multiple objects when a state changes.
Strategy	Defines a family of algorithms, encapsulates them, and makes them interchangeable.

Pattern	Purpose
Command	Encapsulates a request as an object.
Mediator	Centralizes complex communication between objects.
Memento	Captures an object's state to restore later.
State	Changes behavior based on internal state.
Template Method	Defines a skeleton algorithm structure in a base class.
Iterator	Provides a way to access elements of a collection sequentially.
Visitor	Adds new operations to existing objects without modifying them.

Example: Observer Pattern (Notifies multiple objects of state changes)

- When you need a **one-to-many dependency** (e.g., event listeners, messaging systems).

```
import java.util.ArrayList;
import java.util.List;

// Observer interface
interface Observer {
    void update(String message);
}

// Concrete Observer
class User implements Observer {
    private String name;

    public User(String name) { this.name = name; }

    @Override
    public void update(String message) {
```

```
        System.out.println(name + " received notification: " + message);
    }
}

// Subject class
class Channel {
    private List<Observer> subscribers = new ArrayList<>();

    public void subscribe(Observer observer) { subscribers.add(observer); }
    public void unsubscribe(Observer observer) { subscribers.remove(observer); }

    public void notifyObservers(String message) {
        for (Observer observer : subscribers) {
            observer.update(message);
        }
    }
}

// Usage
public class ObserverPatternDemo {
    public static void main(String[] args) {
        Channel techChannel = new Channel();

        User user1 = new User("Nilesh");
        User user2 = new User("Prashant");

        techChannel.subscribe(user1);
        techChannel.subscribe(user2);

        techChannel.notifyObservers("New video uploaded!"); // Notifies all subscribers
    }
}
```

Common Design Patterns in Java Built-in Classes

Design Pattern	Java Built-in Class Example	Explanation
Singleton	<code>Runtime.getRuntime()</code>	Ensures a single instance of <code>Runtime</code> .
Factory Method	<code>Calendar.getInstance()</code>	Returns a subclass based on locale/time zone.
Builder	<code>StringBuilder</code> / <code>StringBuffer</code>	Allows step-by-step construction of mutable strings.
	<code>Stream.Builder</code>	Constructs Java streams flexibly.
Prototype	<code>Object.clone()</code>	Creates object copies using cloning.
Adapter	<code>java.util.Arrays.asList()</code>	Converts array to <code>List</code> (adapts different data structures).
	<code>InputStreamReader</code>	Converts <code>InputStream</code> to <code>Reader</code> .
	<code>OutputStreamWriter</code>	Converts <code>OutputStream</code> to <code>Writer</code> .
Decorator	<code>java.io.BufferedReader(new FileReader("file.txt"))</code>	Enhances <code>Reader</code> with buffering capabilities.
	<code>Collections.synchronizedList(list)</code>	Wraps list with synchronization.
Observer	<code>java.util.Observer</code> & <code>Observable</code> (Replaced by Event Listener)	Implements observer pattern for notifications.
	<code>PropertyChangeListener</code>	Used in JavaBeans for event handling.
Strategy	<code>Comparator</code> (with <code>Collections.sort()</code>)	Allows different sorting strategies.
	<code>ThreadPoolExecutor</code>	Supports different execution strategies via policies.
Command	<code>Runnable</code> & <code>Callable</code> (Used in <code>ExecutorService</code>)	Encapsulates actions as objects.
Proxy	<code>java.lang.reflect.Proxy</code>	Creates dynamic proxies for interfaces.